

Informe de Refactorización con Patrones de Diseño

Equipo Turing

1. Introducción

El objetivo de esta refactorización ha sido desacoplar la lógica de negocio de las rutas de la aplicación (app.py), mejorar la mantenibilidad del código y facilitar la escalabilidad futura. Para ello, se han implementado cinco patrones de diseño de tipo Creacional, encargados de abstraer el proceso de instanciación de objetos.

2. Patrones Creacionales Aplicados

A continuación se detallan los patrones implementados en los archivos patterns.py y models.py.

A. Singleton

Clase: AppConfig (en patterns.py)

- Justificación:

Necesidad de gestionar la configuración de rutas de subida de archivos de manera centralizada.

Problema: Múltiples partes de la aplicación necesitaban acceder a las rutas, y la inicialización de directorios podría ejecutarse múltiples veces innecesariamente.

Solución: Garantiza una única instancia de configuración. Asegura que los directorios se verifiquen una sola vez y proporciona acceso global.

B. Factory Method

Clase: PaymentFactory (Creador) y PaymentProcessor (Interfaz)

- Justificación:

El sistema requiere procesar pagos de diferentes tipos (Tarjeta, Bizum, PayPal) y es escalable.

Problema: Tener lógica condicional (if/else) dentro de las rutas de Flask hace que el código sea difícil de mantener y viola el principio Open/Closed.

Solución: Encapsula la decisión de creación. El cliente solicita un procesador pasando el nombre del método sin conocer la lógica interna de cada pasarela.

C. Builder

Clase: EventoBuilder (en patterns.py)

- Justificación:

La creación de un objeto Evento es compleja: requiere datos básicos, conversión de fechas, precios y gestión de ficheros de imagen.

Problema: La ruta /admin/evento/crear estaba sobrecargada con lógica de validación, conversión de tipos y guardado de archivos en disco.

Solución: Permite construir el objeto paso a paso. Limpia la ruta de administración y encapsula la lógica "sucia" de gestión de archivos.

D. Abstract Factory

Clase: TransactionFactory, EventTransactionFactory, RaffleTransactionFactory

- Justificación:

La compra de una entrada y la de un boleto son transacciones similares pero generan familias de productos distintos (con o sin QR).

Problema: Familia Evento genera Entrada + QR físico. Familia Rifa genera Boleto sin token físico.

Solución: Permite tratar ambas compras de forma polimórfica en el controlador, garantizando la coherencia entre los objetos creados (DB Record + Access Token).

E. Prototype

Clase: Evento, Rifa (método clone() en models.py)

- Justificación:

Necesidad administrativa de crear eventos o rifas recurrentes idénticos a uno anterior salvo por la fecha.

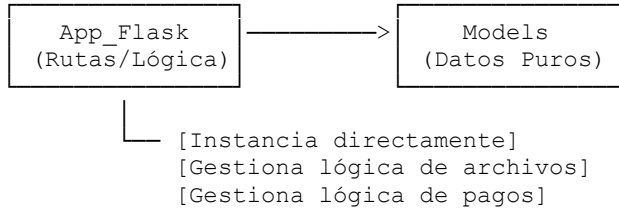
Problema: Tener que rellenar todo el formulario manualmente cada vez es ineficiente y propenso a errores.

Solución: Permite copiar una instancia existente en memoria, lista para ser modificada levemente y guardada como nueva.

3. Diagramas de Clases (Refactorización)

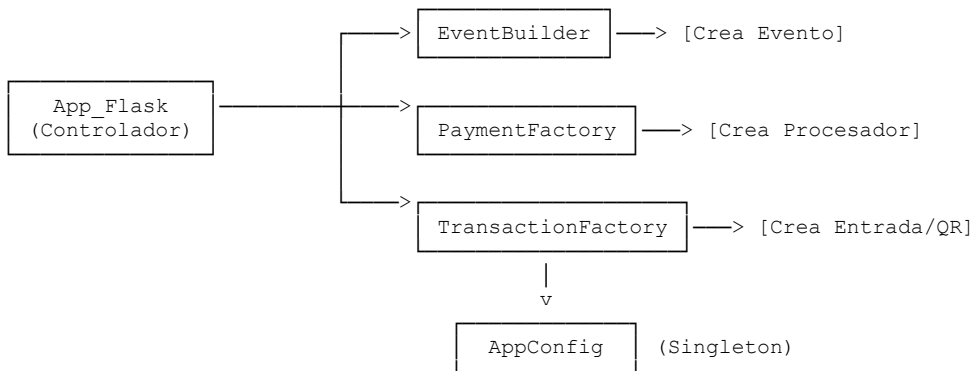
Antes de la Refactorización (Diseño Monolítico)

En este escenario, app.py asumía toda la responsabilidad e instanciaba directamente los modelos, generando alto acoplamiento.



Después de la Refactorización (Patrones)

app.py actúa ahora como un cliente ligero, delegando la creación a las fábricas y builders definidos en patterns.py.



4. Conclusión

La aplicación de estos patrones ha transformado el sistema de un script procedimental a una arquitectura orientada a objetos robusta. Las responsabilidades están claramente separadas (Single Responsibility Principle) y añadir nuevas funcionalidades, como un nuevo método de pago o tipo de evento, ahora requiere cambios mínimos sin alterar la lógica central.